WEEK 1 · LESSON 4 OF 24 · HANDOUT

Testing Machine Learning Systems

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=CU0C4MtrQMU — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon DE 2023: Staying Alert — How to Implement Continuous Testing for Machine Learning Models — https://www.youtube.com/watch?v=4BgmBtsS0iA

🎓 **Go deeper:** EuroPython 2020: Painless Machine Learning in Production — https://www.youtube.com/watch?v=8Zn4hV9xE1I

Your goal in this lesson: Write automated tests for data, model quality and model behaviour, and run them with pytest so problems are caught before users see them.

What can go wrong — and which test catches it

Problem	Test
A column is missing or has impossible values	**Data validation** test
A new model is worse than the old one	**Quality gate** (minimum AUC)
The model behaves strangely: more late payments → more likely to repay	**Directional** test
Tiny, meaningless changes flip the decision	**Invariance** test
Training is not repeatable	**Reproducibility** test (Lesson 2)
The API accepts bad input	**API** test (Lesson 3)

Accuracy alone is not enough: a model can score well and still behave in ways that are wrong or unfair.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Data validation

```
EXPECTED = {
    "income_rwf": (10_000, 20_000_000), "loan_rwf": (10_000, 5_000_000), "years_member": (0,
60),
    "late_payments": (0, 50), "salaried": (0, 1), "momo_tx_month": (0, 1000), "repaid": (0, 1),
}

def validate(df):
    problems = []
    for col, (low, high) in EXPECTED.items():
        if col not in df.columns:
            problems.append(f"missing column {col}")
            continue
        if df[col].isna().any():
            problems.append(f"{col} has {df[col].isna().sum()} missing values")
        bad = ((df[col] < low) | (df[col] > high)).sum()
        if bad:
            problems.append(f"{col} has {bad} values outside {low}..{high}")
    if len(df) < 500:
        problems.append(f"only {len(df)} rows")
    return problems

print("clean data:", validate(applicants))
broken = applicants.drop(columns="momo_tx_month").head(300).copy()
broken.loc[broken.index[:5], "income_rwf"] = -1
print("broken data:", validate(broken))
```

Run validation **before** training and on new data before scoring. Libraries such as Great Expectations and pandera do the same job at scale.

Behaviour tests

```
base = pd.DataFrame([{"income_rwf": 300_000, "loan_rwf": 250_000, "years_member": 4,
                      "late_payments": 1, "salaried": 1, "momo_tx_month": 35}])

def p_repay(df):
    return float(model.predict_proba(df[FEATURES])[0, 1])

print("base:                 ", round(p_repay(base), 3))
print("3 more late payments: ", round(p_repay(base.assign(late_payments=4)), 3))
print("loan doubled:         ", round(p_repay(base.assign(loan_rwf=500_000)), 3))
print("5 more member years:  ", round(p_repay(base.assign(years_member=9)), 3))
print("income + 1 RWF:       ", round(p_repay(base.assign(income_rwf=300_001)), 3))
```

- **Directional:** more late payments and a bigger loan should **lower** the probability; more years of membership should **raise** it.
- **Invariance:** adding 1 RWF to income should change almost nothing.

These tests come from domain knowledge. A model that fails them is suspicious, even with a high AUC.

Turn checks into pytest tests

pytest finds functions whose names start with `test_` and runs them. We save the model and test data, write a test file, and run pytest:

```
import joblib
joblib.dump(model, "model.joblib")
test.to_csv("test_data.csv", index=False)

tests = '''
import joblib
import pandas as pd
from sklearn.metrics import roc_auc_score

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
model = joblib.load("model.joblib")
data = pd.read_csv("test_data.csv")
base = pd.DataFrame([{"income_rwf": 300000, "loan_rwf": 250000, "years_member": 4,
                      "late_payments": 1, "salaried": 1, "momo_tx_month": 35}])

def p(df):
    return model.predict_proba(df[FEATURES])[:, 1]

def test_no_missing_values():
    assert not data[FEATURES].isna().any().any()

def test_quality_gate():
    assert roc_auc_score(data["repaid"], p(data)) >= 0.80

def test_more_late_payments_lowers_probability():
    assert p(base.assign(late_payments=4))[0] < p(base)[0]

def test_bigger_loan_lowers_probability():
    assert p(base.assign(loan_rwf=500000))[0] < p(base)[0]

def test_tiny_income_change_does_not_matter():
    assert abs(p(base.assign(income_rwf=300001))[0] - p(base)[0]) < 0.001

def test_probabilities_are_valid():
    assert ((p(data) >= 0) & (p(data) <= 1)).all()
'''
with open("test_model.py", "w") as f:
    f.write(tests)

import subprocess, sys
result = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider",
"test_model.py"],
                        capture_output=True, text=True)
print(result.stdout[-400:])
```

All **6 passed**. In a real project these tests run automatically every time someone changes the code or retrains the model (Week 2, CI/CD).

A test that catches a bad model

Train a "broken" model on shuffled labels — like a bug that mixes up rows — and run the quality gate again:

```
from sklearn.utils import shuffle
bad_labels = shuffle(train[TARGET], random_state=1).values
bad_model = make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000)).fit(train[FEATURES], bad_labels)
joblib.dump(bad_model, "model.joblib")

result = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider",
"test_model.py"],
                        capture_output=True, text=True)
print(result.stdout.strip().splitlines()[-1])
joblib.dump(model, "model.joblib")                # put the good model back
```

Two of the six tests fail — including the quality gate — so the broken model would never reach users. Notice that the other four still pass: one test is never enough, you need several kinds.

Key terms

- **Data validation** — checking columns, types, ranges and missing values.
- **Quality gate** — a minimum score a model must reach to be released.
- **Directional test** — a change in an input should move the prediction in a known direction.
- **Invariance test** — a meaningless change should not change the prediction.
- **pytest** — the standard Python test runner.

Remember this: Test data, quality and behaviour automatically; a model that passes accuracy but fails behaviour tests should not be released.

Name: _____ Date: _____

Activity

Task 1

Add a test that a salaried applicant has a higher probability than the same applicant without salary.

Task 2

Add a data test that `loan_rwf` is never more than 50 times `income_rwf`.

Task 3

Raise the quality gate to 0.95. What happens, and is that a sensible gate?

Task 4

Write an API test file `test_api.py` for Lesson 3's app with three tests (good request, bad request, health).

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why is accuracy alone not enough to test a model?
   - A. Accuracy is always wrong
   - B. A model can score well but still behave strangely or unfairly
   - C. Accuracy is too slow
   - D. Tests are not needed
2. What does a directional test check?
   - A. The colour of charts
   - B. Speed
   - C. File names
   - D. That changing an input moves the prediction in the expected direction
3. What is an invariance test?
   - A. A fairness law
   - B. The model never changes
   - C. Testing on Sundays
   - D. A meaningless change, like +1 RWF, should not change the prediction
4. What is a quality gate?
   - A. A minimum score a model must reach before release
   - B. A firewall
   - C. A door
   - D. A type of chart
5. How does pytest find tests?
   - A. It needs a GPU
   - B. It reads comments
   - C. It runs functions whose names start with test_
   - D. It guesses
6. What happened when the model trained on shuffled labels was tested?
   - A. pytest crashed
   - B. Accuracy went up
   - C. The tests failed, so it would not be released
   - D. All tests passed

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.